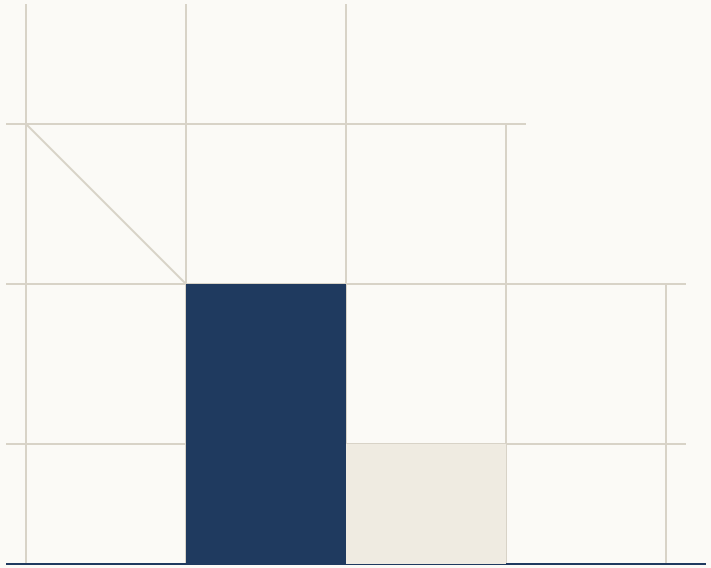
以一些範例來討論 AI Coding 新時代的團隊開發

我與人類夥伴，以及 Agent Team，應該如何重新設計分工



陳宜昌 YC Chen

Noopher AI 創辦人暨執行長

應對的層次，從一個人擴大到整個團隊

People Team + Agent Team

One Person + Agent Team

One Person + AI

兩個自身案例

- 和李宏毅老師合作的語音聊天機器人
- 在 Noopher 打造 AI Native 公司的經驗

來自大佬的經驗



Matt Pocock
Total TypeScript 創作者



Boris Cherny
Claude Code 創作者



Dax Raad
OpenCode 共同創辦人



Simon Willison
Django 共同創作者



Silen Naihin
曾參與 AutoGPT 開發



DHH
Ruby on Rails 創作者



Andrej Karpathy
OpenAI 創始成員



Om Patel
多 agent 工作流分享者



Kieran Klaassen
Every，Compound Engineering



Steve Yegge
前 Amazon、Google 工程師

從帶領研發團隊到創業



陳宜昌 YC Chen

— 8 年以上機器學習研發
— 8 篇 AI 論文（ICLR、ACL 等）
— 語言、語音、視覺模型

研究者 · 聯發創新基地 資深技術經理

帶領團隊開發 Breeze 系列語言模型

台灣第一個繁體中文語言模型

創業者

Noopher AI 創辦人暨執行長

在台灣做技術核心的 AI 新創，用研發改變知識整合與傳遞的方式

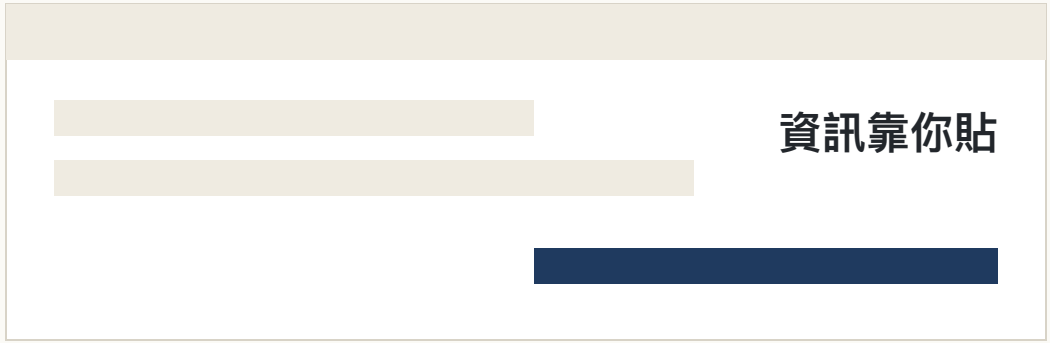
02

一個人＋AI

One Person + AI

從聊天框到 Coding Agent

2025 年 4 月 · 聊天框



只有問題剛好定義清楚、又不用貼太多資訊，
才適合用聊天框寫程式

> **所以不會當成日常：**
> 每次都要貼資訊、組織 prompt，太難了

2025 年 11 月底 · CODING AGENT

Claude Code Opus 4.5

```
my-repo/
  ├─ src/          讀檔、改檔
  ├─ tests/        跑指令、看結果
  └─ README.md
```

AI 直接在你的工作目錄裡協作，
不再只看你貼給它的東西

一用嚇一跳，發現自己晚了至少兩三個月

用共學社群追上，整理出入門三件事

OPEN VIBE DEVELOPERS 線上共學社群

8 場

邀請走在前面的開發者分享

300–400

每場線上參與人數

之後在 COSCUP 2025 主議程演講分享



- 規格先行
- 測試驅動開發
- 照顧好 LLM 的 Context

來源：YC | COSCUP 2025 主議程演講〈Open Vibe Developers：Vibe Coding 的最佳實踐〉（YouTube：YC不只加速器）

把 ICLR 論文做成即時語音聊天機器人

ICLR 2026 論文錄取

TASTE：讓語音與文字對齊

和李宏毅老師合作，把它落地成
即時對話、有情緒變化的語音聊天機器人

工程複雜度

模型 改成 streaming、中文化、對話 post-training

後端 streaming、多使用者、動態調配 GPU

沒有 AI Coding，這些工程會做得非常慢

我＋兩位實習生，8 個月



TASTE2 project page · gitycc.github.io/TASTE2-Homepage

來源：TASTE（ICLR 2026），arXiv 2504.07053；TASTE2 project page（2026-10-07 擷取）

spec 寫到 5,000 行，一次生成仍跑不動

1 · 失敗

5,000+

行 spec，寫了一個月

> 一次生成程式碼
> 跑不動，
> 也不知道錯在哪

2 · 回到原理

照顧好 LLM 的 Context

context 要短、要聚焦
在要解決的問題上

細節多到一個程度，
AI 反而看不懂要做什麼

3 · 回到軟體工程：先定架構



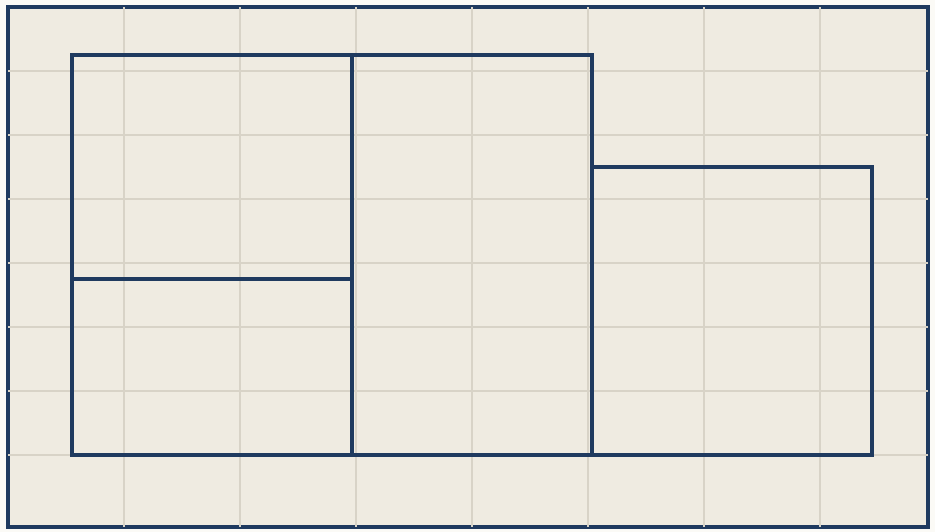
> 一個元件一個元件建起來，
> 再組合在一起

一兩天就跑通

來源：YC｜iThome 專訪

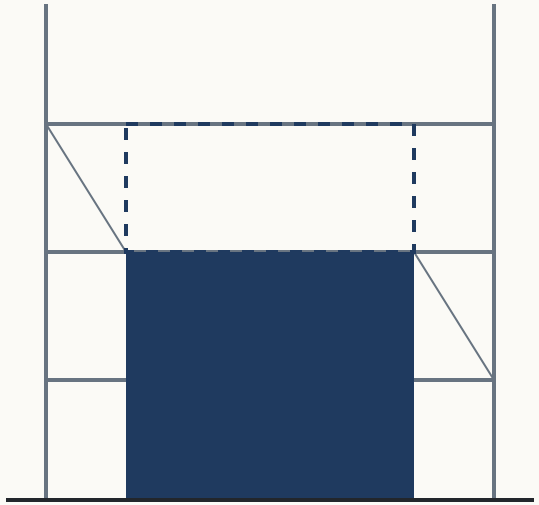
spec 是鷹架，不是藍圖

藍圖



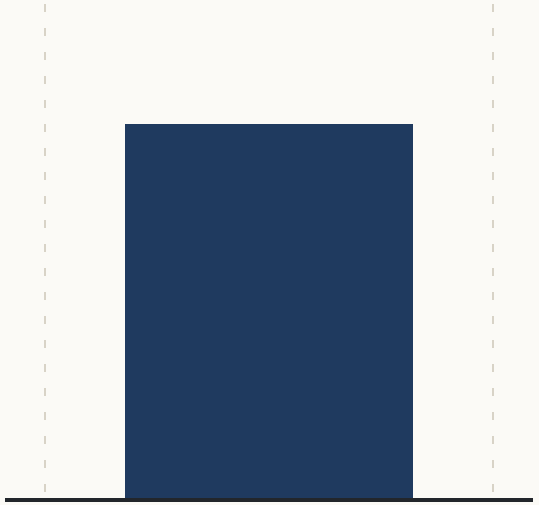
事先想清楚所有細節，照圖施工

鷹架：搭起來



為了蓋一塊，暫時搭起來

蓋好就拆



蓋好，鷹架收起來

03

他山之石

海外開發者看法最不同的兩件事

爭論一：先有規格，還是先有原型？



Matt Pocock

Total TypeScript（英國）

先對齊，再規格

用 Grill Me 讓 AI 問到
形成共同理解，再寫 PRD、
切成可驗證的小功能。

規格不是會自動產出
正確程式的編譯器



Boris Cherny

Claude Code 創作者

先做原型

產品方向用可操作的原型
來探索，取代長篇 PRD；
實作前仍反覆調整計畫。

做出來的東西，
比文件更能幫你決定方向

來源：Matt | AI Engineer 工作坊；Boris | Pragmatic Engineer 訪談整理

爭論二：AI 寫的程式碼，人要看多少？



Simon Willison

Django 共同創作者

用程式碼導覽，把理解補回來

生成之後，用一份逐步的程式碼導覽
把理解補回來，再檢查 agent 建的系統。



Silen Naihin

曾參與 AutoGPT 開發

少看逐行，多管委派和驗收

非同步交出完整任務，減少逐行 review，
把注意力放在委派和驗收。



DHH

Ruby on Rails 創作者

信任多少，要看是什麼系統

新專案高度依賴 AI；但 Basecamp 的局部
變更合起來曾損害架構。



Andrej Karpathy

OpenAI 創始成員、前 Tesla AI 總監

專業工作，仍要監督與可驗證

vibe coding 適合拋棄式專案；專業工作是
agentic engineering。

來源：Simon、Silen｜本人文章；DHH｜Lex Fridman #501；Karpathy｜本人文章

04

指揮 Agent Team

One Person + Agent Team

Noopher AI 的做法：人定架構，逐張單對齊再實作



每張單 · 對齊 用 Matt 的 grill-with-docs，讓 AI 跟我把細節對齊，寫成 spec

每張單 · 實作 用 TDD 把這張單蓋起來，蓋完就關掉 GitHub issue，spec 收起來

來源：YC | iThome 專訪

對齊的核心：人和 Agent 說同一套領域語言



用 Matt 的 skill 落實
`grill-with-docs`

Agent 一題一題問

把每個概念都釐清

問到問題變瑣碎，就同步了

CONTEXT.md

統一的詞彙表

同一個概念只用一個名字

ADR

Architecture Decision Record

架構決策紀錄：決定了什麼、為什麼

背後的概念：DOMAIN-DRIVEN DESIGN · 領域驅動設計

程式的結構和命名，跟著業務領域的概念走

開發者、領域專家和 Agent 用同一套語言（Ubiquitous Language）溝通，同一件事不會各說各話

來源：Matt Pocock | grill-with-docs、domain-modeling skills；頭像取自 GitHub

plan、執行、review 的迴圈，佔滿我的注意力

拆單、寫完 spec 之後，每張單都要走一遍



我看一下計畫書，
通常沒什麼要改

它問我問題：太複雜看不懂，我同意；
方案看起來像我要的，我也同意

單元測試 → 整合測試 → spec review（逐條驗收）
→ 變異測試：把 > 改成 <，測試應該轉紅燈

看一下報告，再回到 plan，直到測試乾淨

非常制式，我卻一直坐在電腦前等它

我想要的

它照流程自己走，
真的卡住，
才來回報我。

我的注意力，留給
真正能發揮的地方

v1：meta-agent 拆單，subagent 平行執行

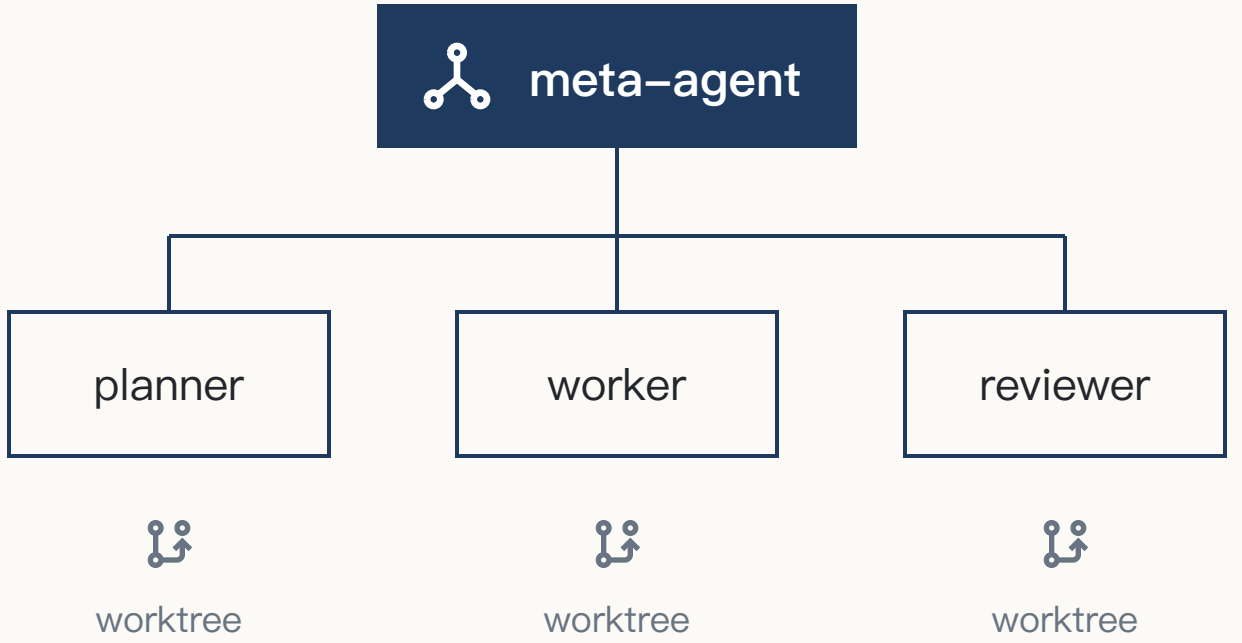
做法

我只對 meta-agent 下指令，
它去調度 subagent

Claude Code 原生 subagent

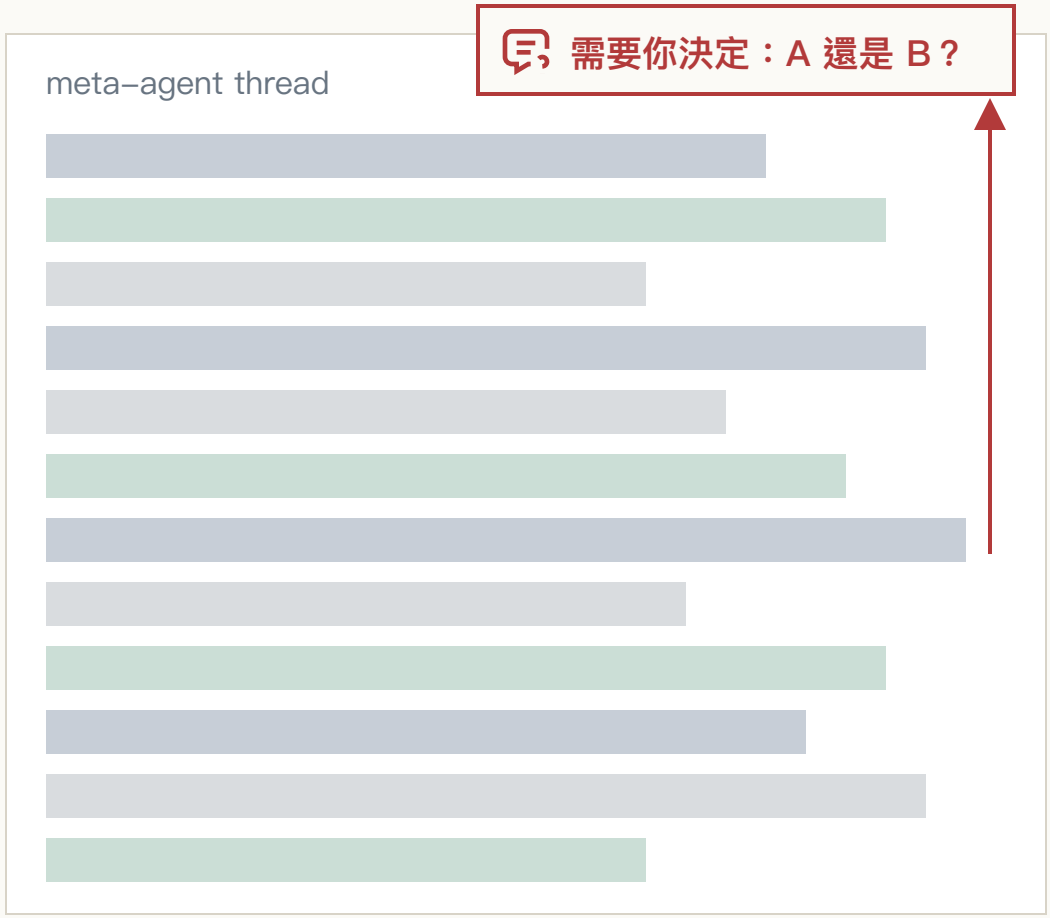
先拆單，用 worktree 平行派工

過程寫紀錄，包成 skill



平行度很快，隔著 meta-agent 看問題

v1 的瓶頸：一條 thread 承載所有資訊



卡在哪

看不出現在跑的是哪張單

需要我決策的地方被洗上去

沒人回答，流程卡死

一條 thread 就是全部的 UI

示意圖：三種顏色是三張單的訊息交錯

v2：用看板流把進度變得看得見



每個 agent 醒來先看看板，用 CLI 操作

視覺化解決了

示意圖；當時搭配一個看板流的開源工具

v2 的代價：Agent 多背一套看板規則



看板本身成了每個 agent 的額外負荷
任務越複雜，越容易出錯

為了把它跑順

1 個月

十億等級 token

修 prompt、遇到問題、再修
沒完沒了

推估：Max 5x 方案 Opus 週上限約 4.5 億 token（撞到上限那週的實測）× 約 4 週

回到第一性原理：要的是好的檢視介面，不是看板

	瓶頸		所以需要
v1	資訊沒辦法好好被呈現、被接收	→	**人看得清楚 Agent 的脈絡**
v2	資訊管住了，Agent 的認知負荷卻太大	→	**Agent 做事簡單，沒有額外負荷**

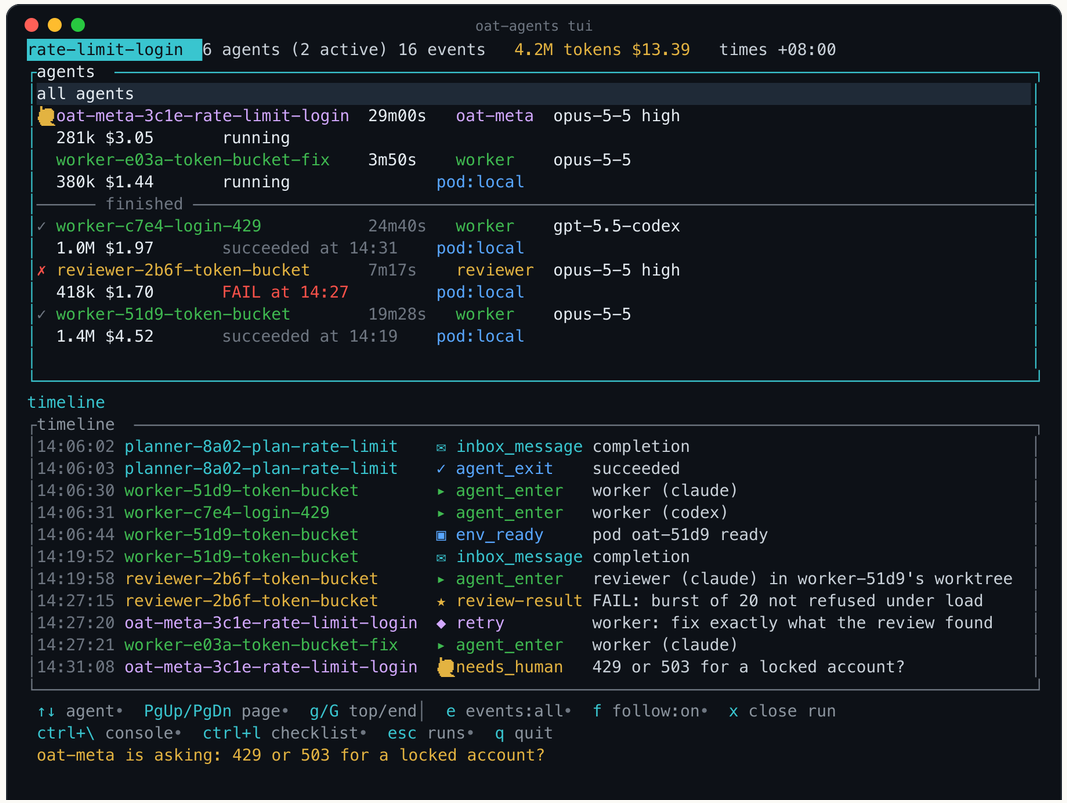
我需要看板嗎？

不需要。只要 v1 能好好被觀察、被檢視就夠了

v3 要同時做到：結構簡單 · Agent 沒有額外負荷 · 人看得見脈絡

v3：oat-agents，用最精簡的結構做到這三件事

找遍開源專案，沒有一個襯手的，所以自己開發，並開源給大家



① 結構簡單

meta-agent 只帶一層 member agent；
團隊、派工邏輯與 Skill 都用 plugin 客製

② Agent 沒有額外負荷

member agent 就是原生 Claude Code / Codex，
worktree 檔案管理與 pod 隔離交給工具

③ 人看得見脈絡

只用 TUI 呈現資訊流，一個畫面看整隊；
用 tmux 連進任一 session，直接下指令

畫面取自 oat-agents README 的示範回放；來源：github.com/Noopher-AI/oat-agents

05

People Team + Agent Team

人的團隊與 Agent Team 一起工作

先看清楚：人擅長什麼？Agent 擅長什麼？

AI Agent 做得好的事

- 執行力高、效率高
- 細心穩定，重複性工作表現佳
- 適合流程化、機械式、低價值判斷的任務

AI Agent 做不好的事

- 容易和人的真正目標不對齊
- 容易卡在細節，看不到大局
- 技術選型兩難時，較難跳脫框架

結論：AI 強在執行；人強在全局決策，以及做好與 AI 的對齊

實作交給執行團隊，人組成決策團隊

執行團隊 AGENT TEAM

- 實作
- 測試
- 除錯
- 第一輪 review

成果 →

← 方向

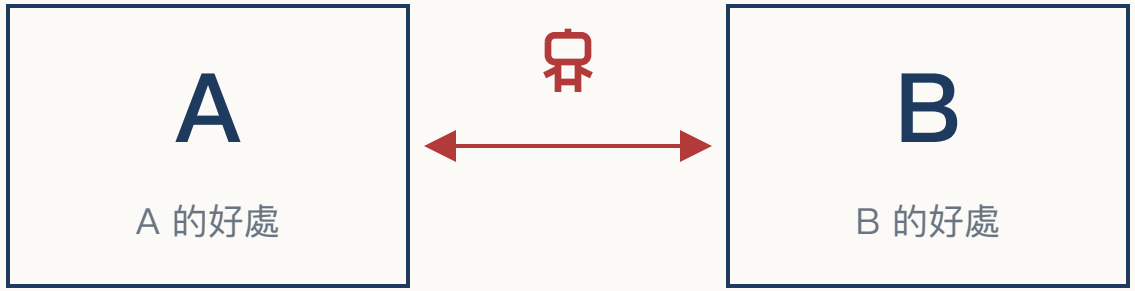
決策團隊 PEOPLE TEAM

決策

技術選型、架構方向與取捨

Agent 在選項間打轉，人負責跳出框架

Agent 的框架



選哪一邊，都要放棄另一邊

問它有沒有兩者兼得的解法，它答不出來

人跳出框架

第三種解法

同時保有 A 與 B 的好處

技術選型與架構方向，
要握在人手上

實作交給執行團隊，人組成決策團隊

執行團隊 AGENT TEAM

- 實作
- 測試
- 除錯
- 第一輪 review

成果 →

← 方向

決策團隊 PEOPLE TEAM

決策

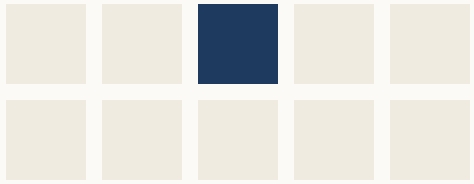
技術選型、架構方向與取捨

理解工程

跟上 AI 的產出，抓住影響大的地方

新人一進團隊，就得理解整個系統

我當新人時



全公司的程式碼複雜到看不懂，
老鳥告訴我這裡大概怎麼改，我就在那個範圍裡做事

現在的新人



AI 替你實作，你反而要更懂全局：
一進來就得懂整個系統

好消息是，AI 也讓你比以前更快懂

實作交給執行團隊，人組成決策團隊

執行團隊 AGENT TEAM

- 實作
- 測試
- 除錯
- 第一輪 review

成果 →

← 方向

決策團隊 PEOPLE TEAM

決策
技術選型、架構方向與取捨

理解工程
跟上 AI 的產出，抓住影響大的地方

探索工程
帶著 AI 找更好的可能

探索更好的可能，再疊加回原型

理解系統

先知道它現在長什麼樣子

帶著 AI 探索

收集網路上的資訊

和 AI 討論更好的做法



疊加回原型，讓它更好

然後再理解、再探索

實作交給執行團隊，人組成決策團隊

執行團隊 AGENT TEAM

- 實作
- 測試
- 除錯
- 第一輪 review

成果 →

← 方向

決策團隊 PEOPLE TEAM

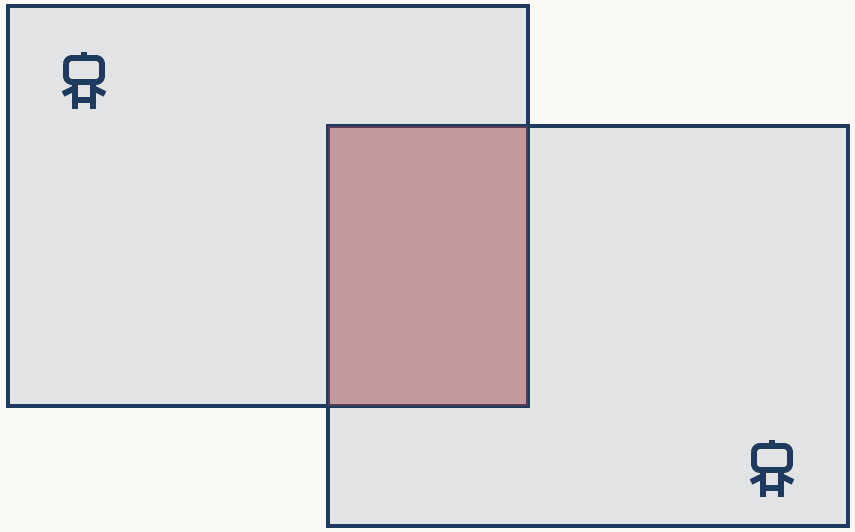
決策
技術選型、架構方向與取捨

理解工程
跟上 AI 的產出，抓住影響大的地方

探索工程
帶著 AI 找更好的可能

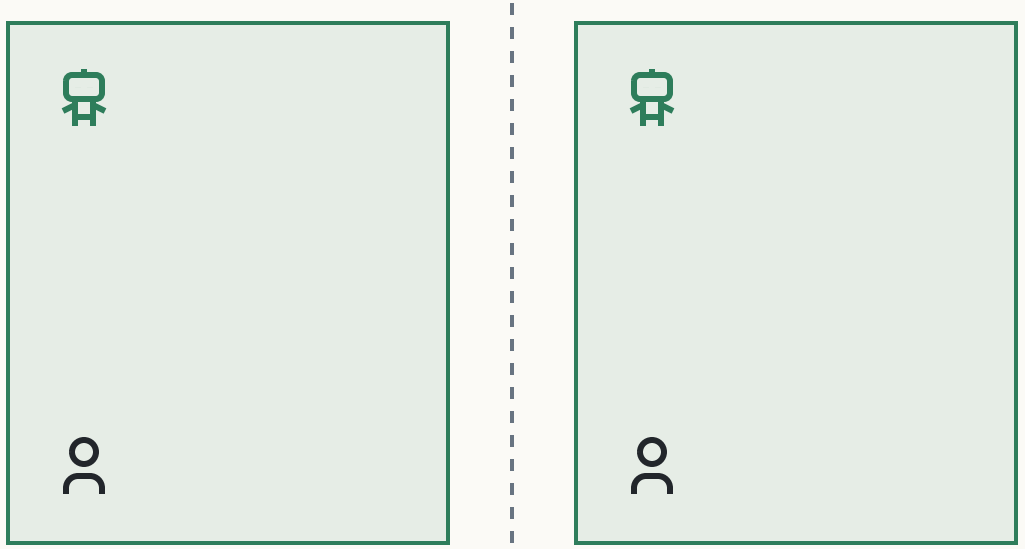
每個人都帶著 Agent，scope 要切得很開

scope 重疊



速度太快，互相踩
merge request 多到處理不完

scope 切開



各自的 scope，各自的 Agent

原型期守住三個關卡，幾乎不看程式碼

Agent 的產出流過三個關卡

驗證

怎麼證明它是對的

介面

模組之間的約定

use case

我們要怎麼使用它

三關卡住，AI 就不會錯得太離譜。

適用範圍：原型快速疊代期。我們看的是三個關卡，和 AI 整理出的高影響改動

AI Native 公司要注意的三件事

1

拆分執行與決策團隊

Agent Team：執行

People Team：決策

各司其職

2

好的 Agent 執行團隊

這套系統要能：

- 持續運行
- 被優化、會學習
- 容易和人對齊
- 隨時看得懂目前狀態

3

人的課題

- 定義目標：對標市場、客戶、成果
- scope 大幅拆開來合作
- 學會全盤理解

重心從親手做，
變成持續對齊、持續決策

AI 替你實作，人反而要更懂整個專案

06

收尾

帶回去的三句話

三個層次，各帶走一句話

今天的主軸

我們要因為 AI Coding，重塑既定的工作流。

1

One Person + AI

spec 當鷹架，
蓋完就收起來

2

One Person + Agent Team

Agent 只管做事，
人看得見脈絡

3

People Team + Agent Team

實作交給 AI，人負責
理解、探索、決策

One More Thing: AI 時代最重要的是判斷力

決定做什麼

眼光與品味的判斷力

培養獨到的眼光、獨特的品味，
看見別人看不到的機會

過程怎麼做

找出真正訊號的判斷力

從 AI 的雜亂資訊中找出真正的訊號，
在過度發散的組合裡找到最佳決策

而這一切，都瓶頸於認知

用 AI 加速學習 → **拓寬認知**

用 AI 快速試錯 → **快速獲取經驗**

這是一個充滿機會的時刻，想把握機會，你得跑起來

歡迎找我交流，也歡迎一起開發 oat-agents

了解 YC



陳宜昌 YC Chen

Noopher AI 創辦人暨執行長

掃描看我的數位名片

想聊 AI Coding 與團隊開發，歡迎找我交流

了解 oat-agents

Noopher-AI/oat-agents

MIT 開源，Rust 單一執行檔，一行指令安裝

掃描前往 GitHub

歡迎發 issue、開 PR，一起來開發

數位名片：noopher.com/m/ycchen | GitHub：github.com/Noopher-AI/oat-agents